

中山大學

SUN YAT-SEN UNIVERSITY

# 并行程序设计与算法实验

## Lab0-环境设置与串行矩阵乘法


姓 名 林宏宇

学 号 23320093

学 院 计算机学院

专 业 计算机科学与技术


2026 年 4 月 9 日

# 1 实验目的

- 理解并行程序设计的基本概念与理论。
- 掌握使用并行编程模型实现常见算法的能力。
- 学习评估并行程序性能的指标及其优化方法。

# 2 实验内容

- 设计并实现以下矩阵乘法版本：
  - 使用 C 语言实现一个串行矩阵乘法。
  - 比较不同编译选项、实现方式、算法或库对性能的影响：
    - 使用 Python 实现的矩阵乘法。
    - 使用 C 实现的基本矩阵乘法。
    - 调整循环顺序优化矩阵乘法。
    - 应用编译优化提高性能。
    - 使用循环展开技术优化矩阵乘法。
    - 使用 Intel MKL 库进行矩阵乘法运算。
- 生成随机矩阵 A 和 B，进行矩阵乘法运算得到矩阵 C。
- 衡量各版本的运行时间、加速比、浮点性能等。
- 分析不同实现版本对性能的影响。

# 3 实验结果

## 3.1 基础实验结果

表中各列定义如下：

- 相对加速比：相对于表中前一版本的加速比（$T_{prev}/T_{curr}$）。
- 绝对加速比：相对于版本 1（手工 Python）实现的加速比（$T_{v1}/T_{curr}$）。
- 浮点性能 (GFLOPS)：按照总浮点运算数 $2 \times m \times n \times k$ 除以运行时间计算得到，再除以 $10^9$。
- 峰值性能百分比：GFLOPS 与设备理论峰值 GFLOPS 的比值。

本次实验实现了多个版本、多个规模下的实验结果对比，下面给出 $(m, n, k) = (1024, 1024, 1024)$ 的结果示例：

| 版本 | 实现<br>描述 | 运行时间<br>(s) | 相对<br>加速比 | 绝对<br>加速比 | 浮点性能<br>(GFLOPS) | 峰值性能<br>百分比 |
|---|---|---|---|---|---|---|
| 1 | Pure Python (for) | 76.605955 | 1.000000 | 1.000000 | 0.028 | 0.00 |
| 2 | C (-O0) | 3.764368 | 0.049139 | 20.350283 | 0.570 | 0.02 |
| 3 | 调整循环顺序 (ijk) | 3.230137 | 0.042166 | 23.716008 | 0.665 | 0.02 |
| 4 | 编译优化 (-O3) | 0.345789 | 0.004514 | 221.539594 | 6.210 | 0.20 |
| 5 | 循环展开 | 0.369832 | 0.004828 | 207.137172 | 5.807 | 0.19 |
| 6 | Intel MKL | 0.045228 | 0.000590 | 1693.772767 | 47.481 | 1.55 |

## 3.2 进阶实验结果

## 3.3 不同版本的对比分析

在 $(m, n, k) = (1024, 1024, 1024)$ 的详细对比表：

| 版本 | 实现<br>描述 | 运行时间<br>(s) | 相对<br>加速比 | 绝对<br>加速比 | 浮点性能<br>(GFLOPS) | 峰值性能<br>百分比 |
|---|---|---|---|---|---|---|
| 1 | Pure Python (for) | 76.605955 | 1.000000 | 0.0002 | 0.028033 | 0.00 |
| 2 | Python (numpy) | 0.0135266 | 5663.3688 | 1.0000 | 158.760398 | 5.17 |
| 3 | C basic (-O0) | 3.764368 | 0.0036 | 0.0036 | 0.570477 | 0.02 |
| 4 | C (i-j-k) | 3.230137 | 1.1654 | 0.0042 | 0.664827 | 0.02 |
| 5 | C optimized (-O3) | 0.345789 | 9.3414 | 0.0391 | 6.210387 | 0.20 |
| 6 | 循环展开 | 0.369832 | 0.9350 | 0.0366 | 5.806646 | 0.19 |
| 7 | C tiled (64) | 0.271351 | 1.3629 | 0.0498 | 7.914044 | 0.26 |
| 8 | C tiled + SIMD | 0.269236 | 1.0079 | 0.0502 | 7.976213 | 0.26 |
| 9 | Intel MKL / BLAS | 0.045228 | 192.489896 | 0.299075 | 47.481287 | 1.545615 |

## 3.4 C-O0 与 C-O3 在不同规模对比

| 尺寸 | 实现<br>描述 | 运行时间<br>(s) | 相对<br>加速比 | 绝对<br>加速比 | 浮点性能<br>(GFLOPS) | 峰值性能<br>百分比 |
|---|---|---|---|---|---|---|
| 512x512x512 | C (-O0) | 0.469440 | 1.000000 | 1.000000 | 0.571821 | 0.018614 |
| 512x512x512 | C (-O3) | 0.043756 | 10.728586 | 10.728586 | 6.134826 | 0.199701 |
| 1024x1024x1024 | C (-O0) | 3.715118 | 1.000000 | 1.000000 | 0.578039 | 0.018816 |
| 1024x1024x1024 | C (-O3) | 0.349006 | 10.644854 | 10.644854 | 6.153142 | 0.200298 |
| 1536x1536x1536 | C (-O0) | 12.587298 | 1.000000 | 1.000000 | 0.575799 | 0.018743 |
| 1536x1536x1536 | C (-O3) | 1.227772 | 10.252146 | 10.252146 | 5.903179 | 0.192161 |
| 2048x2048x2048 | C (-O0) | 30.238304 | 1.000000 | 1.000000 | 0.568149 | 0.018494 |
| 2048x2048x2048 | C (-O3) | 3.992449 | 7.573874 | 7.573874 | 4.303090 | 0.140075 |

# 4 实验分析

- 在 $(1024, 1024, 1024)$ 下，Pure Python 运行时间为 76.605955 s，GFLOPS 仅为 0.028；切换到 C（`-O0`）后时间降至 3.764368 s，绝对加速比达到 20.350283，GFLOPS 提升到 0.570，体现出解释执行与编译执行在同一算法上的明显差异。

- 在基础实验的 C 实现内部，循环顺序调整（ijk）将时间从 3.764368 s 降至 3.230137 s，相对前一版本提升约 1.165×，GFLOPS 从 0.570 提升到 0.665，说明仅通过访存模式调整已可获得稳定收益。

- 编译优化（`-O3`）将时间进一步降至 0.345789 s，相对上一版本加速约 9.34×，相对 Python 基线绝对加速约 221.54×，GFLOPS 达到 6.210。该阶段是基础实验中单步增益最大的环节。

- 循环展开版本时间为 0.369832 s，略慢于 `-O3` 版本（0.345789 s），对应 GFLOPS 由 6.210 降至 5.807，说明在该数据规模与实现条件下，手工展开未进一步超过编译器自动优化结果。

- Intel MKL 版本时间为 0.045228 s，相对上一版本再提升约 8.18×，相对 Python 基线绝对加速约 1693.77×，GFLOPS 达到 47.481。从基础实验的整体序列看，性能提升路径呈现为：语言与编译方式切换 → 访存与循环优化 → 高性能库调用。

- 从“C-O0 与 C-O3 在不同规模对比”可见，`-O3` 在 512、1024、1536 规模下加速比分别约为 10.73×、10.64×、10.25×，到 2048 下降至 7.57×；同时 `-O3` 的 GFLOPS 由约 6.13 降到 4.30，表明随着规模增大，计算过程中的存储层次开销占比上升。